A3-PS022-TC127

eBPF/XDPRate Limiting for DNS Query Floods in a Lab

BYTEGUARD

Prashant Sengar | Janhvi Gokhale

Faculty Mentor: Dr. Nitish Andola

Indian Institute of Information Technology, Design and Manufacturing, Jabalpur

AIORI-3 · Level 1 Proposal • October 12, 2026

Problem Description

- High-rate UDP DNS queries consume CPU and buffer resources through kernel packet processing, socket queues, and DNS application handling. Much of this work can be wasted on unwanted requests.
- A single-source rate limit cannot constrain aggregate traffic from many individually low-rate senders. Large numbers of source addresses can also create pressure on bounded source-tracking state.
- A strict global limit can protect server capacity but also reject legitimate queries during congestion. The filtering mechanism must remain inexpensive while minimizing unnecessary drops.

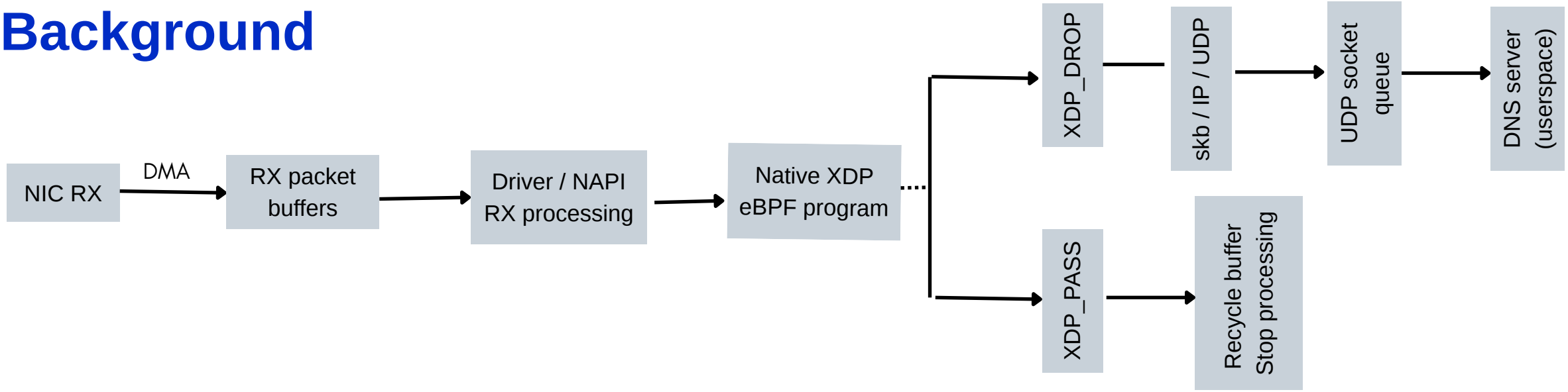
PROJECT OBJECTIVE

Design an XDP-based DNS admission filter that bounds per-source and aggregate traffic while keeping packet-processing overhead low.

Evaluate CPU utilization, throughput, packet drops, and legitimate-query success under three controlled DNS flood scenarios, compared against no-filter and nftables baselines.

Scope: isolated Linux host pair · inbound IPv4/UDP DNS queries · no public flood testing

Background



GLOBAL ADMISSION BOUND

The DNS server has finite processing capacity. Our global token bucket limits the total number of admitted DNS queries, regardless of how many source IPs participate.

$$N(T) \le Bg + RgT$$

Bg = allowed initial burst (queries)

Rg = configured refill rate (queries/sec)

N(T) = queries admitted during interval T

WHY PER-SOURCE + GLOBAL LIMITS?

- Single-source flood: One sender can consume excessive capacity; a per-source limit constrains it.
- Distributed flood: Many sources can each remain below their individual limits; the global bucket bounds their aggregate load.
- Legitimate traffic: A global limit alone can also reject genuine queries under saturation; preserving client success remains a measured trade-off.

Native XDP acts before ordinary skb allocation; generic XDP does not. Test mode will be recorded.

Literature & RFC Review

The existing building blocks we are using — and what each contributes.

Source	Relevant lesson	How it affects our proposal
RFC 1035 [1]	DNS wire format, 12-byte fixed header, query/response bit.	Bounded DNS header checks.
RFC 768 [2]	UDP header fields and datagram length.	Validate target UDP packets safely.
XDP / CoNEXT 2018 [3]	Early programmable packet handling in the Linux kernel.	Keep rejection in the driver fast path.
Linux BPF maps [4]	LRU hash has bounded entries and automatic eviction.	Avoid permanent new-client map lockout.
dnsdist + nftables [5, 6]	Operational per-client limiting, bursts and source lifetime policies.	Use proven rate-limit ideas and a strong baseline.

Our contribution is a measured XDP implementation — not a claim of a new DNS or rate-limiting standard.

Proposed Solution

Our current baseline: bounded DNS admission with source state; refine only when tests justify it.

PASS only when the global budget and the per-source admission both succeed.

TRACKED SOURCE

1. Read-only source eligibility check
2. Reserve one global token
3. Atomic GCRA source commit
4. On commit failure: refund global token

NEW SOURCE

1. Reserve global token before insertion
2. Insert in bounded LRU map (BPF_NOEXIST)
3. On insert race: use existing-source path
4. On failure: refund global token

IMPLEMENTATION CHOICES

IPv4/UDP/53 bounded parser · LRU_HASH source state · atomic 64-bit GCRA · synchronized global bucket

Global admission: at most B + R×T queries in any interval T, for the covered traffic. LRU eviction may reset source history; it does not bypass the global bound.

Optimization Proposed

Design comparison — expected behavior, not measured speed-up.

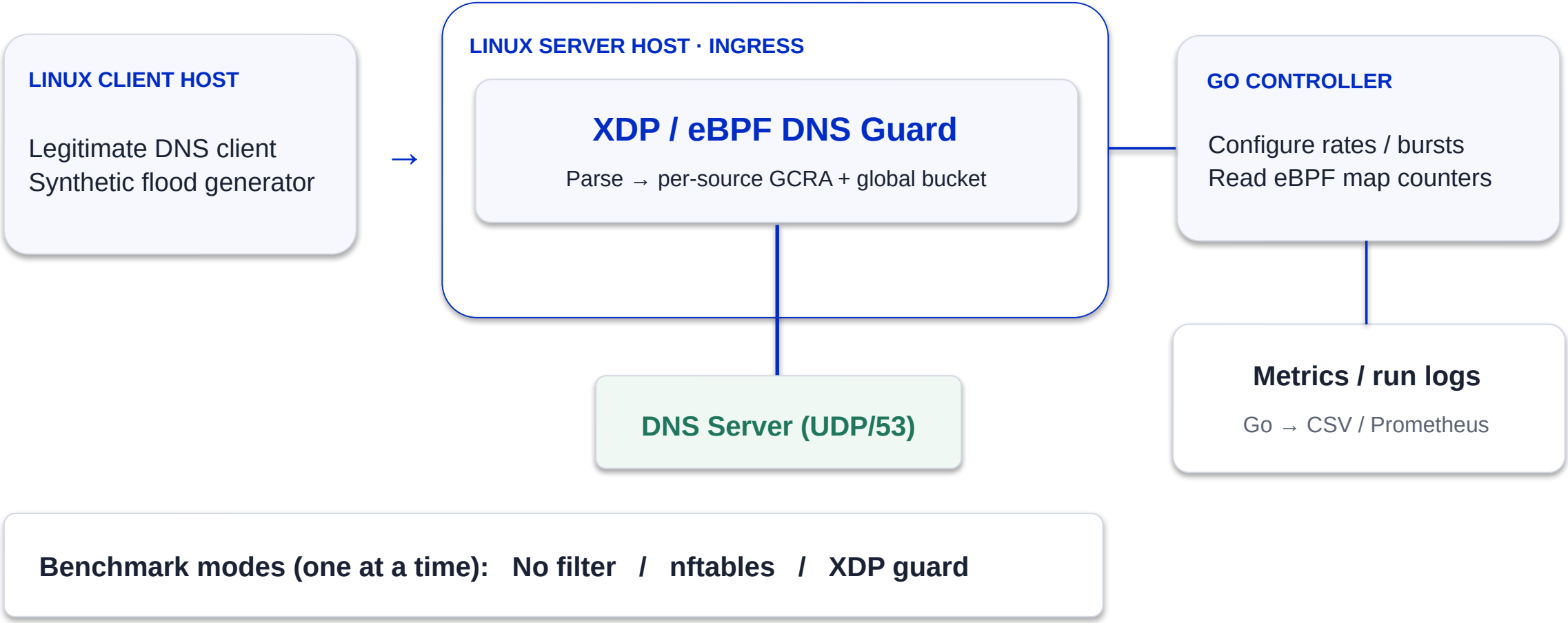
Area	Straightforward implementation	Our selected / proposed design
Parsing	Decode full DNS question	Check bounded IPv4/UDP + fixed DNS header
State lifetime	Fixed HASH can fill forever	Bounded LRU permits controlled replacement
First-packet cost	Create state before admission	Global admission before new-source insertion
Source limiting	Spin lock + refill arithmetic	Atomic GCRA timestamp (CAS); bounded retries
Global accounting	Premature global token deduction	Reserve globally; refund a failed source commit
Telemetry	Shared counters / per-packet logs	Per-CPU counters; aggregate in Go

Engineering checks: CAS: bounded retries · refunds: synchronized · LRU: eviction-race tests

Baseline for implementation; adapt source handling or global budgets if tests show a need.

Solution Architecture & Design

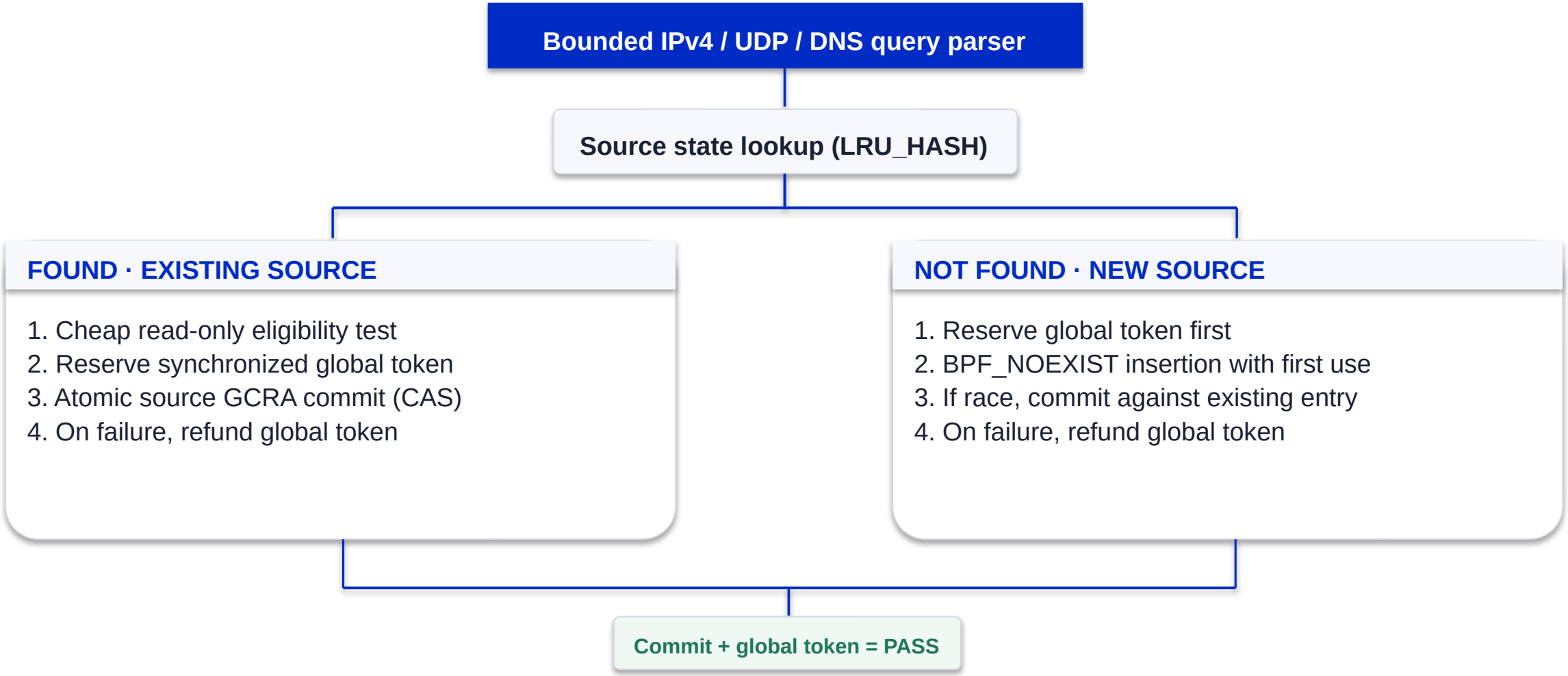
One Linux host pair in an isolated DNS-flood lab.



Native XDP preferred; if generic XDP is used, its mode and limitations will be reported.

Data Flow Diagram

Only validated target DNS queries enter the limiter; other traffic follows the documented scope policy.



Otherwise DROP. LRU eviction may reset source history; global budget remains authoritative.

Deployment Timeline

From proposal to a reproducible, comparable working demonstration.

12–14 Oct	**Level 1 proposal**	Submit PDF + team video; verify repo; aim for XDP parser / loader smoke test.
16–19 Oct	**Data plane + controller**	XDP parser, Go loader; check CAS/LRU races, global refunds and verifier acceptance.
20–23 Oct	**Controlled traffic harness**	Linux host pair, DNS server, legitimate stream and three flood patterns.
24–27 Oct	**Fair baseline comparison**	Run identical workloads with no-filter, nftables and XDP; collect CPU, QPS and drops.
28–31 Oct	**Results and delivery**	Repeat tests, analyze legitimate success/latency, finalize report, source and demo.

Reproducibility: same traffic seed, duration, offered QPS and query mix; record kernel, XDP mode and policy config.

Bibliography I

Selected sources that directly shaped the proposal.

[1] P. Mockapetris, RFC 1035 (1987)

DNS header structure and query/response semantics.

https://www.rfc-editor.org/info/rfc1035

[2] J. Postel, RFC 768 (1980)

UDP header format used in bounded parsing.

https://www.rfc-editor.org/info/rfc768

[3] Høiland-Jørgensen et al., CoNEXT (2018)

The eXpress Data Path: fast packet processing.

https://doi.org/10.1145/3281411.3281443

[4] Linux kernel documentation

BPF HASH and LRU map semantics / eviction.

https://docs.kernel.org/bpf/map_hash.html

[5] PowerDNS dnsdist documentation

Per-source DNS QPS, burst and expiry rules.

https://www.dnsdist.net/reference/yaml-selectors.html

[6] nftables wiki, Meters

Dynamic per-source rate limits and timeouts.

https://wiki.nftables.org/wiki-nftables/index.php/Meters

[7] Linux kernel documentation

Per-CPU counter maps and synchronization.

https://docs.kernel.org/bpf/map_array.html

[8] AIORI-3 Hackathon Guidelines

Official Level 1 deliverables and milestones.

https://portal.aiori.in/aiori-3/hackathon-guideline/



Thank You for Your Attention!

Protect the DNS service. Keep every decision inexpensive. Measure the impact on users.

BYTEGUARD · A3-PS022-TC127 · IIITDM Jabalpur

Level 1 proposal — performance results to follow after controlled experiments.